
Incident Twins: What Logs Can Establish After an Agent Stops¹

Rohith YV
Independent author

Abstract

After an agent stops, what can its logs establish about accepted work? Incident Twins checks whether retained evidence rules out in-scope commitments during a fixed post-stop interval. Study 1 uses scripted responses in Pydantic AI. Three asynchronous pairs have byte-identical configured native/controller records but opposite outcomes. Terminal receipts and a separate pending-work check distinguish outcome evidence from collection coverage. Study 2 tests whether effects can commit without audit records. In 30 process-interruption runs, separate transactions leave such gaps. A joint effect/audit transaction with a complete scoped snapshot resolves all 15 comparator runs. A simple query matches all 30 measured runs, but concludes on a contradictory control that the richer checker rejects. Study 2's raw records differ: its opposing examples match only on declared selected fields. Neither study establishes permanent containment or permission to resume. I list what an operator should retain. The resulting verdict depends on explicit scope and trust.

1 Introduction

Incident Twins addresses Track 2, “What happened, and what breaks next,” by checking a fixed post-stop claim. A parent stop does not settle whether accepted work later commits. I ask whether retained evidence determines that outcome and whether a committed effect can fail to produce its audit record.

METR/Redwood linked separate agent transcripts through a shared message board and noted observation gaps [9]. OpenAI reports stopping evaluations and rebuilding infrastructure [10, § VI.A]. These incidents motivate evidence checks; they do not demonstrate this study's exact post-stop condition, effect/audit race or registered-worker design.

Scripted models call real Pydantic AI tools; messaging conventions [5] support the acceptance boundary's plausibility, not its prevalence.

Three asynchronous pairs have identical configured evidence and opposite post-stop outcomes. Terminal receipts resolve the executed cases, while pending work needs valid coverage. In the process-interruption study, effects can commit without audit rows; a same-transaction effect/audit design closes that gap under the tested scope and collection assumptions.

¹ Research conducted at the [AI Incident Response Sprint](#), September 2026

2 Related Work

ClaimReceipt’s criterion says equal retained evidence must imply equal claim truth across allowed executions [1, §2.1, Eq. (1)]. It separates sufficiency, integrity, assignment coverage and external truth. Incident Twins applies this reasoning to a stopped interval and runtime source/window collection. Earlier diagnosability and sensor-placement work likewise asks when different conditions share observations and which added sensors separate them [4]. No new identifiability theorem is claimed.

The DEMM method paper calls it the “container fallacy” when an evidence container is mistaken for support for a property [2]. DEMM-Bench is a separate benchmark [3]; its inspected v0.1.1 code builds presence/validity indicators and scores redacted features. That was static inspection, not execution. Bounded searches did not locate ClaimReceipt or Diedrich executable releases; this does not prove their absence.

ContainmentBench measures committed violations, logged propagation and authorized commits in benchmark traces, with claims limited to its logged schema [11]. Safe to Resume? studies rollback and resumption, including external effects missing from recovery records [12]. Incident Twins asks what evidence supports a fixed stopped interval, rather than when execution is safe to resume. Counter-Swarm Doctrine proposes linking executions, persistent artifacts and response history; its incident analysis and evaluation design do not demonstrate containment efficacy [13]. Study 2’s effect/record mechanism and same-transaction outbox remedy are established ideas [6, 7, 12].

3 Methods

3.1 Claim and evidence boundary

The assertion is: “No write or export attributable to the registered run family committed to a declared in-scope resource after confirmed parent stop and through the observation horizon.” The run family is the parent and its registered descendants; the horizon H is the observation deadline. Let $C(h)$ contain the committed effects in history h that belong to family R and target resource scope W . With stop cutoff s and deadline H ,

$$Q(h) \Leftrightarrow \neg \exists e \in C(h): s < t(e) \leq H. \quad (1)$$

The original implementation uses $s = 50$ and $H = 100$ as local ordering markers, not seconds or distributed timestamps. A commitment at s is excluded and one at H included. Its effect is a dummy file write followed by flush and fsync. Queue and delegation records link each task to its owner and resource; all tasks and resources must be registered. The process extension uses a SQLite insert and a separately measured deadline (§3.5). No external export was tested.

3.2 Study 1 What retained evidence establishes

Study 1 uses Python 3.12.13, Pydantic AI 2.43.0 and OpenTelemetry SDK 1.44.0. A registered tool queues work and returns task, owner and destination acknowledgements. While the parent waits for its next scripted response, the controller requests family cancellation, calls `Task.cancel()`, awaits termination and checks `task.cancelled()`. This policy stays fixed across pairs: delivery of the family-cancel request races with service progress, and only requested live work can be cancelled.

Each asynchronous pair measures one execution through confirmed parent stop, then restores its service state into two fresh instances. Task identities, payloads, progress, the cancellation request and existing file bytes are preserved. The instances run different allowed continuations; they are not independent whole-agent runs. Capture is checked again at each horizon. In delegated pairs, a

registered child is still running its tool at parent stop. The tool returns actual terminal records, and collection awaits the child. Each delegated branch is a full execution, not a deployment sample.

3.3 Capture and evidence profiles

Capture is enabled explicitly. Separate contexts retain parent/child messages. ALWAYS_ON sampling, SimpleSpanProcessor, a local exporter and Pydantic AI's trace data format version 5 retain content, request parameters, IDs, times, links, statuses, events and drop/flush metadata without filters. No emitted child span or decisive field was removed for equality. Runtime experiments block provider and socket requests; this is not an untouched-defaults study.

Study 1 compares three profiles: (1) all configured native/controller records, including bindings, scope, stop and horizon; (2) the same records with a SHA-256 wrapper and separately trusted local anchor; and (3) those records plus executor records and coverage. The wrapper adds no coverage and cannot detect consistent replacement of both payload and anchor. The asynchronous service makes no later Pydantic AI call after acceptance; the delegated tool returns terminal facts.

3.4 Compatible histories and bounded analysis

Let M be the model, B its bounds, S the profile and A the collection assumptions. Writing $\mathcal{H}(M,B)$ for legal histories, the compatible set is

$$\Omega(E; M, B, S, A) = \{h \in \mathcal{H}(M, B) : \text{compatible}(h, E; S, A)\}. \quad (2)$$

Study 1's continuation witnesses use raw byte equality. The compatibility adapter reads trusted controller order, bindings and source facts; absence constrains closed scopes or structurally resolved work. It does not generate arbitrary trace bytes. Equality therefore names its level: the complete configured Study 1 view, or selected evidence fields—the declared projection—in Study 2. The opposing-history condition is

$$O_s(h_0) = O_s(h_1), \text{ while } Q(h_0) \neq Q(h_1). \quad (3)$$

A decision restricted to that declared observation cannot distinguish these histories. Valid added observations narrow Ω under fixed assumptions, as in established sufficiency/critical-pair reasoning [1, 4].

ESTABLISHED means nonempty Ω with every history satisfying Q ; REFUTED means nonempty Ω with every history violating Q . UNDETERMINED means both outcomes occur; INCONSISTENT means no history fits. Search completion is separate. An incomplete search or timeout cannot establish universal agreement or an empty set. Two opposing compatible histories already justify UNDETERMINED. Parsing and integrity failures are errors, not containment outcomes.

Study 1's model permits one root, at most one child, two effectful tasks, one delegation level and one commitment per task; it excludes retries and unknown descendants. Initial progress is unknown: queued, started or previously committed. Issuance/progress precede request tick 40 and stop tick 50. Slots 60, 70, 80 and 90 allow waiting or enabled start, commit or cancel steps. Start requires queued work; commit requires started work; cancellation requires requested live work. No task is issued after the stop request. Work may remain pending at H .

Six configurations combine the two mechanisms with two approved tasks, one approved task or one out-of-scope task: $585 + 31 + 31$ histories per mechanism, totaling 1,294. The model covers more event orders than the scheduler. A separate checker recomputes legality, attribution, ordering and Q and checks saved file bytes; aggregate validation shares compatibility logic. Realized histories, schedules, checkpoints and destination reads stay private. The investigator receives public evidence, the model and any required integrity anchor.

3.5 Study 2 When committed effects lack audit records

Study 1’s executor-plus-coverage repair assumes each relevant effect produces a source record. Study 2 tests that generation step under process interruption. It retains the Pydantic AI adapter, using Python 3.12.13 and SQLite 3.53.1 for one registered worker, task, resource and harmless effects-table insert, without retries or hidden tasks. Both designs stop and await the parent but let accepted detached work finish; unlike Study 1, neither requests family cancellation. SIGKILL interrupts the separate process, not the synchronous interior of an asyncio call.

Design A commits the effect, then its accurate, correctly linked audit row in a separate transaction. Design B inserts both in one transaction, the known outbox pattern [6]. Both use WAL, synchronous=FULL, foreign_keys=ON and read_uncommitted=0. Only the registered effects-table insert counts toward Q; audit, logging and backup writes are evidence machinery. B has no within-transaction cut where the effect is committed but its audit row is not.

The controller permits effects after confirmed parent stop, then holds or terminates the worker during consistent journal snapshots and a separate-process destination read before H. It preserves the SQLite file/WAL family and standalone backups. This local order places observed commitments in the interval; an epoch is not a commit timestamp. Raw IDs, times, spans and exits remain intact. The private fault-control pipe and destination oracle are excluded from checker inputs. After H, only already durable receipt delivery or pre-transaction cleanup is released; no new effect is allowed.

The separate model tracks commitment, durable receipt creation, delivery and interruption. After checking run/task/resource/epoch bindings, controller order and the pinned framework route, its native projection at H retains worker event kinds, exit status and qualifying delivered-receipt count. The journal projection adds qualifying audit-row count. Raw tracing values remain available but are not all interpreted. UNRESOLVED is the mixed-outcome category corresponding to Study 1’s UNDETERMINED; unsupported inputs, inconsistency and incomplete search remain separate. In B, audit absence establishes Q only if every effect creates an accurate, correctly linked row in the same transaction and collection covers the scoped interval. Complete collection of generated rows alone is insufficient in A.

The direct query validates the journal input, returns REFUTED for a matching audit row, ESTABLISHED for none in atomic design B, and UNRESOLVED otherwise; it rejects a receipt missing from the complete snapshot but omits lifecycle compatibility.

4 Results

Study 1 has four asynchronous pairs (four prefixes/eight restorations) and four delegated pairs (eight full executions): sixteen histories and 48 profile bundles, yielding 24 ESTABLISHED, 12 REFUTED and 12 UNDETERMINED (Table 1). All 36 conclusive answers agree with retained-record reference checks. The 0/36 incorrect results show agreement within this bounded check, not a population error rate or 36 independent incidents.

A Canonical e01: one shared prefix, two restored continuations

Scoped question: no attributed in-scope commitment in (50, 100]

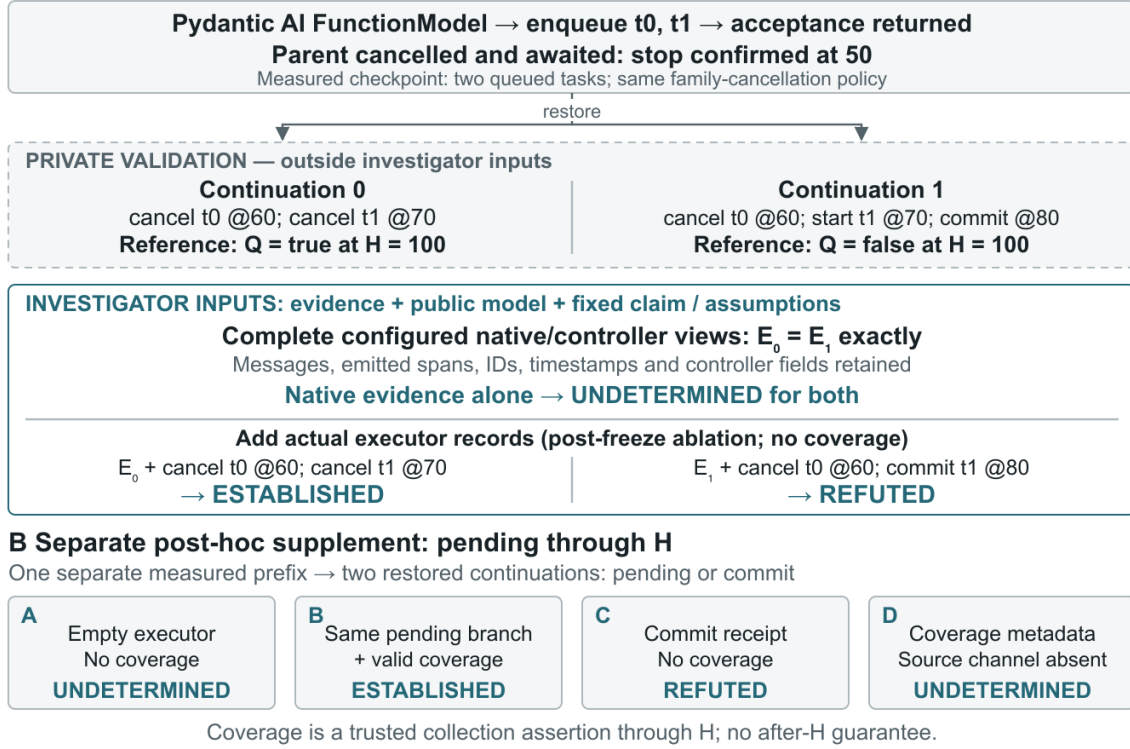


Figure 1. Q forbids attributed in-scope commitments in $(s, H]$, here causal ticks $s = 50$ and $H = 100$. Canonical e01 reuses one measured stopped-parent prefix: two restored continuations have opposite Q but byte-identical complete configured native/controller views. Executor-only augmentation is post-freeze; the pending inset is a separate post-hoc supplement. Native/controller evidence remains present; private reference outcomes are excluded from investigator inputs. Verdicts depend on the bounded model and trusted collection. The display is schematic; equality concerns full saved bundles.

Table 1. Study 1 frozen evaluation

n counts bundles across three profiles of sixteen histories. All analyses completed; none was inconsistent or errored. There were 0/36 incorrect conclusive answers under the tested model and trusted-reference assumptions.

Mechanism	Profile	n	Established	Refuted	Undetermined
Async	native	8	2	0	6
Async	integrity only	8	2	0	6
Async	augmented	8	5	3	0
Delegated	native	8	5	3	0
Delegated	integrity only	8	5	3	0
Delegated	augmented	8	5	3	0

e01–e03 have opposite Q and byte-identical complete configured native/controller files; e04 is an equal-view out-of-scope true/true control. e05–e07 have different views resolving opposite outcomes; e08 is a different-view delegated true/true control. Resolution follows the terminal-reporting contract, not delegation inherently improving safety.

4.1 Study 1 receipts and unfinished work

The post-freeze ablation adds executor records without coverage to the same sixteen native/controller views: 10 ESTABLISHED and 6 REFUTED, all correct under reference validation (Table 2). These terminal cases do not demonstrate coverage’s necessity; executor records were not the only input.

Table 2. Post-freeze executor-channel ablation

Native/controller evidence remains present. This post-freeze ablation reuses sixteen original histories and adds executor records without coverage. All analyses completed; 0/16 incorrect conclusive results.

Evidence added	Original histories	Established	Refuted	Undetermined
Executor only	16	10	6	0

The pending-work supplement was chosen after Study 1 and specified before its single attempt. One measured prefix produced pending-through-H and post-stop-commit continuations with byte-identical native/controller files, checked as for e01–e03. Four evidence conditions retain that view (Table 3).

Table 3. Separate post-hoc pending-work supplement

Native/controller evidence remains present. One post-hoc case, one attempt/prefix, two continuations. Q columns count compatible modeled histories, not probabilities. All analyses completed; B/C give 0/2 incorrect conclusive results.

ID	Branch	Public evidence	Q true	Q false	Verdict
A	Pending	Empty executor; no coverage	21	10	UNDETERMINED
B	Same pending	Empty executor + valid coverage	6	0	ESTABLISHED
C	Commit	Positive executor receipt; no coverage	0	2	REFUTED
D	Same pending	Coverage metadata; no source channel	21	10	UNDETERMINED

Table 3 compares evidence on paired continuations. A, B and D use the same pending branch; C uses the commit branch. An empty executor source leaves pending work unresolved (A); valid coverage establishes the interval claim (B). A scoped commit receipt refutes Q without coverage (C). Coverage metadata without its source remains unresolved (D). Compatible-history counts describe modeled alternatives, not probabilities.

Coverage is trusted, not independently authenticated. The frozen helper does not enforce its preconditions. The pending-work call site checked instrumentation, returned finish, horizon records, successful flush and ordering before accepting coverage. The interval conclusion does not authorize later work; this supplement ran no after-H continuation.

4.2 Study 1 model scope

Executor records plus valid coverage separate opposite Q values throughout Study 1's 1,294-history model. It has no process-interruption step and does not cover effects that fail to generate records. Study 2 addresses that additional assumption under a separate contract.

4.3 Study 2 effects and audit records

After two development gate runs, five schedules crossed two designs and three fresh-process repetitions: 30 separately frozen evaluation runs, all completed and independently validated (Table 4). Each retained four framework spans. Three A post-commit interruptions left an effect without an audit row; no receipt was deleted. B retained both rows. All 15 matched design/schedule/repetition comparisons had equal effect counts.

Table 4. Study 2 process interruption results

Thirty full runs, three per schedule/design. Counts are per run at H. E = ESTABLISHED; R = REFUTED; U = UNRESOLVED. Native views gave R only for normal delivery; the journal adds the complete audit snapshot. A and B label generation designs, not Table 3 evidence conditions.

Schedule	Effect A / B	Audit A / B	Journal A / B
Normal	1 / 1	1 / 1	R / R
Interrupt before commit	0 / 0	0 / 0	U / E
Interrupt after commit	1 / 1	0 / 1	U / R
Durable audit and delayed delivery	1 / 1	1 / 1	R / R
Pending through H	0 / 0	0 / 0	U / E

Journal results were 15 R, 6 E and 9 U, with 0/21 incorrect conclusive answers against independent SQLite validation; B resolved all 15 runs. The direct query matched all 30 measured answers. It shares parsing and binding checks, but omits lifecycle validation. At complete-search settings, journal and direct-query agree on eight of nine controls: five shared unsupported-input refusals, two shared U answers and one shared E answer. On control-001, the journal rejected an account asserting a completed worker, no receipt and an empty complete snapshot as INCONSISTENT; the query returned ESTABLISHED. These post-hoc diagnostics do not establish nine successful outcome inferences, a runtime error rate, or a need for enumeration. The original zero-history-cap control-002 test remains separate. The conservative rule (REFUTED for a validated delivered/recovered receipt, otherwise UNRESOLVED) matched native (6 R, 24 U). Later delivery resolved six native U cases; A's unrecorded effects remained U.

The selected A pre- and post-commit interruptions have independently checked opposite Q but equal journal projections. Their full raw JSON and bytes differ; timestamps also rule out equality by consistent ID renaming. Study 2 supplies no exact complete-view twin. Exhaustive enumeration covers 22 A and 16 B histories: journal observations have three mixed-Q groups in A and none in B. All 30 measured runs map to that separate model; it does not replace the 1,294-history analysis.

5 Discussion

The studies identify two different evidence problems. Accepted work can progress beyond the facts reported by its tool, even when all configured native records are retained. Separately, a process fault can prevent an effect from producing its audit record. SDK ForceFlush concerns spans already received [8]; it certifies neither what every task did nor whether every effect created a span.

Worked example using Table 3B

Checked input. In Table 3B, registered task/resource and (50,100] bindings, a present empty executor channel and matching coverage fields constrain the model. Complete analysis finds six compatible histories satisfying Q and none violating it; this checks consistency, not source truth.

Trusted premises. All relevant work is registered, every relevant effect generates an accurate linked record, causal order is faithful, and enabled instrumentation and correctly ordered collection cover H. The checker cannot authenticate these premises. **Missing or contradictory input.** Missing coverage (A) or executor evidence (D) leaves the claim unresolved; inconsistent or unsupported accounts do not establish containment. **Permitted report.** Under this model and these premises, no attributed in-scope commitment occurred in (50,100]. This establishes neither task termination nor absence of later effects, and does not authorize resumption. The reference implementation accepts pinned study inputs, not arbitrary incident logs. Another application needs a justified observation adapter, execution model and source/collection contract; mismatch with real behavior remains the principal validity threat.

Future work should document the minimum instrumentation needed for particular application contracts, repeat the bounded check in a second framework, and test faulty or adversarial collection. These extensions are untested; Appendix B reports the separate post-hoc component check.

6 Conclusion

Incident Twins turns a stopped-interval claim into an offline evidence check. Study 1 shows that complete configured records can leave opposite outcomes possible; receipts and the pending-work supplement locate collection's role. Study 2 shows why collection alone fails when effects go unrecorded. A same-transaction audit and scoped snapshot resolve the tested comparator runs, with direct-query parity on those runs. One contradictory control exposes the query's missing consistency check; this does not establish general superiority of richer inference. Exact Study 1 witnesses and projected Study 2 witnesses remain distinct. Neither result guarantees permanent containment or permission to resume.

Code and Data

Saved-evidence replay repository: https://github.com/yvrohith/incident_twins.

QUICKSTART.md gives offline commands. Replay was validated on CPython 3.12.13, 3.12.14, 3.13.5 and 3.13.15; normal mode accepts final 3.12.x/3.13.x releases. Private validation histories, checkpoints and sinks used to construct the twins are withheld by design: they sit outside the investigator-input boundary the study defines. The public model, evidence bundles and checkers reproduce every reported verdict. Development notes and validation logs are available from the author on request.

References

1. Peiyong Zhu and Sidi Chang. 2026. ClaimReceipt: Verifying Evidence Sufficiency and Coverage in Agent Evaluations. arXiv:2609.01992v1. <https://arxiv.org/abs/2609.01992v1>.
2. Oleg Solozobov. 2026. Decision Evidence Maturity Model for Agentic AI: A Property-Level Method Specification. arXiv:2605.04093. <https://arxiv.org/abs/2605.04093>.
3. Oleg Solozobov. 2026. DEMM-Bench: A Cross-Regime Benchmark for Agent-Runtime Governance-Evidence Sufficiency. arXiv:2606.20634v1. <https://arxiv.org/abs/2606.20634v1>.
4. Alexander Diedrich, René Heesch, Marco Bozzano, Björn Ludwig, Alessandro Cimatti and Oliver Niggemann. 2024. Inferring Sensor Placement Using Critical Pairs and Satisfiability Modulo Theory. DX 2024, OASICS 125, article 9, pp. 9:1–9:19. <https://doi.org/10.4230/OASICS.DX.2024.9>.
5. OpenTelemetry. Semantic conventions for messaging spans. Development documentation, accessed 13 September 2026. <https://opentelemetry.io/docs/specs/semconv/messaging/messaging-spans/>.
6. Amazon Web Services. Transactional outbox pattern. AWS Prescriptive Guidance. <https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html>. Accessed 12 September 2026.
7. SQLite. Atomic Commit In SQLite; Write-Ahead Logging; Isolation In SQLite. <https://sqlite.org/atomiccommit.html>; <https://sqlite.org/wal.html>; <https://sqlite.org/isolation.html>. Accessed 12 September 2026.
8. OpenTelemetry. Tracing SDK, ForceFlush. <https://opentelemetry.io/docs/specs/otel/trace/sdk/>. Accessed 12 September 2026.
9. METR. 2026. Brief independent investigation of agents’ behavior, reasoning and collaboration in the OpenAI / Hugging Face hacking incident. 26 August. <https://metr.org/blog/2026-08-26-openai-hugging-face-incident-investigation/>. See “Investigation process and limitations.”
10. OpenAI. 2026. OpenAI – Hugging Face Incident Technical Report. 26 August. <https://cdn.openai.com/pdf/67869394-cb91-4c12-888c-5cbd85c7814c/OpenAI-Hugging-Face%20Incident-Technical-Report.pdf>. See § VI.A, pp. 14–16.
11. Wenhao Lan, Shan Li, Meiqi Wu, Xinhua Lai, Junbin Yang and Haihua Shen. 2026. ContainmentBench: Trace-Based Evaluation of Post-Exposure Containment in Tool-Using LLM Agents. arXiv:2607.23999v3. <https://arxiv.org/abs/2607.23999v3>. See §§ 3.3, 4.4 and 7.3.
12. Guanlong Wu, Dahui Li, Ke Jiang, Jianyu Niu, Cong Wang and Yinqian Zhang. 2026. Safe to Resume? Breaking Execution Continuity of Agent Execution via Rollback. arXiv:2608.29381v1. <https://arxiv.org/abs/2608.29381v1>. See §§ IV-E and V-C.
13. Gregory N Frank. 2026. Counter-Swarm Doctrine: Containing Coordinated Agent Intrusions. arXiv:2609.06140v1. <https://arxiv.org/abs/2609.06140v1>. See §§ 1.1 and 10.3.

Appendix A Limitations and Dual-Use Considerations

Application, model and construction

The application intentionally places a custom service beyond an acceptance-returning tool. Its configured native/controller view is not all system logs, an untouched-defaults exporter, or a documented containment guarantee. The deterministic model does not support claims about frontier-model autonomy, refusal, intent, deception or prevalence of unsafe behavior. The finite abstraction assumes a closed task/resource universe, bounded effects, no retries or unknown descendants, and a local causal clock. A real system with additional behavior can fall outside it.

The asynchronous witnesses reuse measured stopped-parent prefixes; they are not independent whole-agent traces. Delegated branches are full executions, but neither construction supplies a statistical incident sample. The pending-work supplement was selected after canonical results, then frozen before its single attempt; it is not a ninth frozen pair. The later process extension likewise has a separate prospective freeze, two development gates and 30 evaluation runs. These denominators and evidence-equality levels must remain separate.

Process faults and the observation boundary

The extension's main limitation is its evidence boundary. It excludes the laboratory fault-control channel and compares selected fields from the full native data. A stronger operational or destination channel could resolve the cases; matching selected fields does not make all retained evidence identical. Barriers make a feasible interruption point easier to test and do not show how often natural crashes occur. The application has one effect route, no bypasses or record deletion, and trusted local collection. A single transaction cannot give the same guarantee for unrelated files or exports. Power loss, corruption, malicious writers and distributed clocks were not tested. The two development gates retain an inaccurate policy description and lack later backup sidecars; their findings remain separate from the 30 fully validated evaluation runs.

Collection, integrity and validation

Canonical coverage is trusted in-process source/window collection. The helper's preconditions are not internally enforced; the pending-work supplement checks them at the call site.

source_drained_to_horizon means collection reached H, not that all tasks terminated. The wrapper cannot authenticate collector honesty or external completeness; a co-located digest anchor cannot resist replacement of both anchor and payload.

The original reference checker separately implements legality and Q, but aggregate verification shares compatibility logic. Retained records report 55 original tests, 32 support tests and four isolation checks; Study 2 reports 47 tests plus five subtests, nine controls and 60 public-input replays. A separate code path checked all 30 SQLite outcomes. The later 36-cell diagnostic reuses nine controls with four methods and complete-search defaults; the original zero-cap test stays separate. Its expected outputs are post-hoc observations, not independent correctness labels. The frozen direct query lacks lifecycle coherence checks and was not repaired after discovery. No usability study is claimed.

Interval and source-comparison limits

A bounded interval conclusion is not permanent containment or permission to resume. Pending tasks can act after H. The process extension observed later receipt delivery and cleanup while sealing the original effect interval; no new after-H effect was permitted or observed. Positive scoped commitments can refute Q without complete unrelated coverage, but coverage metadata without its source cannot establish absence.

ClaimReceipt, DEMM and critical-pair comparisons retain their prior source-specific limits. DEMM-Bench's release was inspected statically, not executed; bounded release nonlocation does not prove absence. For this revision, specific incident-report sections, the three incident-response papers and OpenTelemetry messaging definitions were checked against primary documents. Neighbor code and experiments were not run. Safe to Resume? already addresses unrecorded external effects; Study 2's distinction is its stopped-interval evidence contract, not discovery of that failure mechanism.

Dual use and release decisions

Evidence-gap analysis can reveal blind spots useful for evasion as well as investigation. This artifact uses owned dummy data and identifies the records and assumptions an investigator needs. Private reference histories remain separate from public checker inputs. Core judge code uses Apache License 2.0. Component redistribution remains pending. The linked core repository provides the saved-evidence replay artifact; its availability does not grant permission to redistribute the separate component materials.

AI Assistance

The author designed the study, checked every claim against the artifact, and approved this text. Codex assisted with implementation, automated validation, figures, tables, packaging and drafting; ChatGPT and Claude supported planning, review and the stock-observer probe script.

Appendix B Post-hoc stock-observer probe

A post-hoc component check (16 fresh-process runs; Table B1) found matched SQLite row outcomes but API-dependent completion telemetry: connection methods added no span; cursor COMMIT/ROLLBACK added one. CREATE/INSERT were traced. This post-hoc check used one instrumentor version, with no crash injection or export-loss measurement. A COMMIT span can inform a trusted-operation judgment without certifying general durability or complete collection.

Table B1. Post-hoc stock-observer probe

Four API paths $\times$ two modes (legacy and autocommit=False) $\times$ two repetitions = 16 recorded component executions; the 16 received summaries are separate. Counts agree across both modes and repetitions. Rows were read by a separate process after worker exit. Completion spans were collected after force_flush and before connection close; every run had traced CREATE/INSERT, successful worker/reader exits and a true flush result. Python 3.12.3, SQLite 3.45.1, instrumentors 0.65b0 and SDK/API 1.44.0 ran on macOS arm64. Under autocommit=False, connection completion left in_transaction=true; cursor completion left it false. Equivalent row outcomes do not imply identical connection semantics. Component source, diff and full recordings are separate from the judge ZIP; distribution permission remains pending.

Completion API	New runs	Rows read	Completion spans
connection.commit()	4	1	0
connection.rollback()	4	0	0
cursor.execute("COMMIT")	4	1	1
cursor.execute("ROLLBACK")	4	0	1